Health monitoring Platform

Team members:

- Vurchio Francesco, s327008
- Venturini Marcelo, s338023
- Vole Andrea, s305932

Overview

The Health Monitoring Platform is an IoT-based system designed to monitor both patients' medical parameters and their home environment.

The platform has two main purposes:

1) To simplify the monitoring of certain medical parameters by doctors.

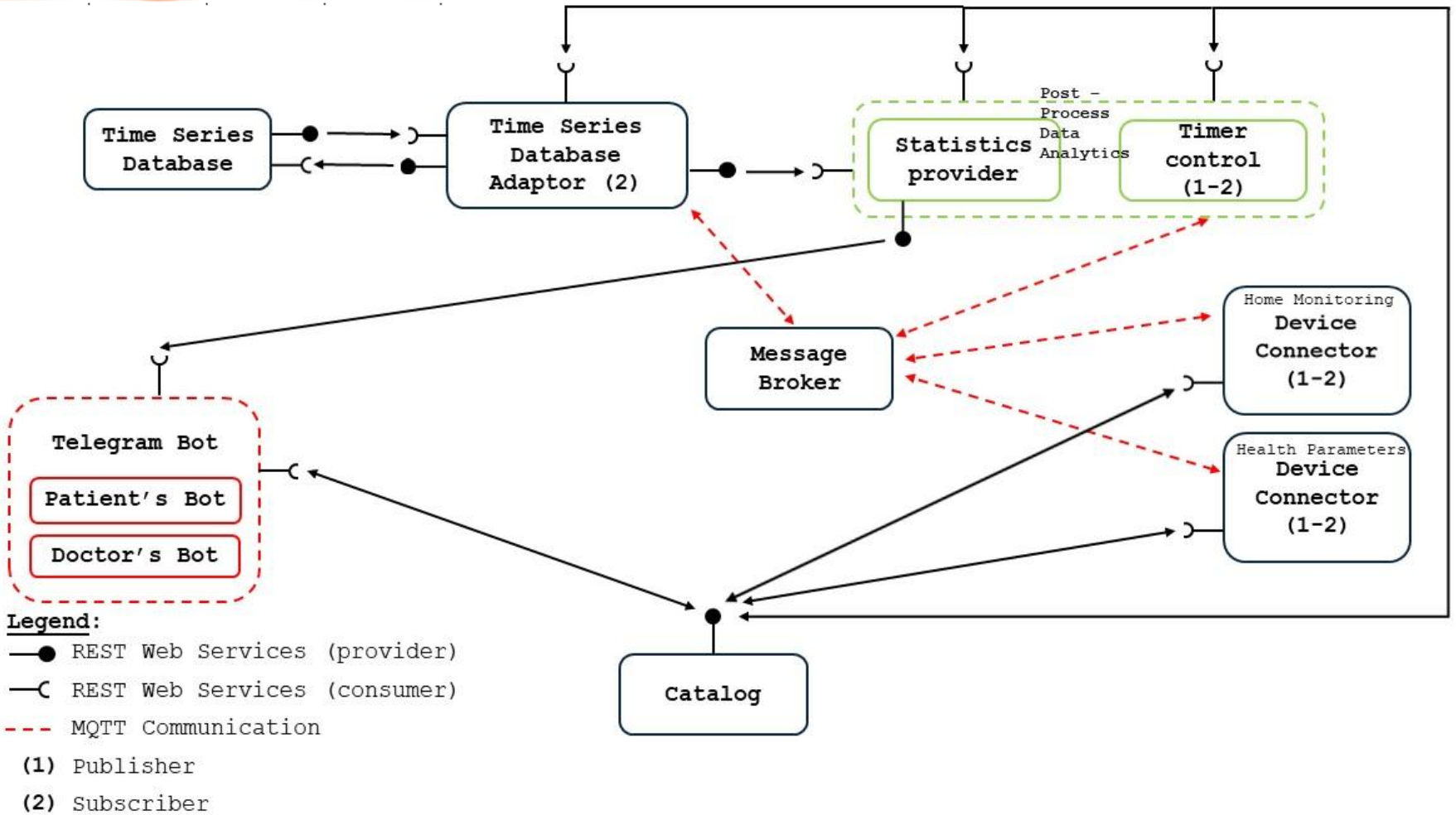
2) To automatically regulate the temperature and humidity levels of patients' rooms.

The first objective has been implemented using simulated sensors that send medical data to ThingSpeak. The data can then be visualized through a Statistics Provider that generates a PDF report, accessible to both patients and doctors.

The second objective is achieved through a dedicated control module that continuously checks whether temperature and humidity remain within acceptable ranges and automatically triggers alerts or actions if anomalies are detected.

As the user interface, we implemented two Telegram bots, one for doctors and one for patients, with an intuitive design that enables real-time interaction and easy access to reports.

Use case diagram



Communication Protocols

REST web services

- Allows **synchronous communication** between microservices
- **Catalog** exposes the information located in JSON file to all microservices
- **Thingspeak adaptor** uploads sensor data and distributes it when requested to
- **Statistic Provider** elaborates data and provides it to the telegram bot for users to access it

MQTT

- Allows **asynchronous communication** between sensor data providers and consumers
- **Device connectors** publish sensor data in SenML format using QoS 2
- **Thingspeak adaptor** subscribes to topics using QoS 2
- **Message topic** hierarchical, contains sensor and patient information

Catalog

- Handles communication with all microservices in the platform
- Employs REST communication

```
> class CatalogREST: ···


def run_cherrypy_server(catalog):
    local_ip="catalog"
    port = 8080
    # Configurazioni di CherryPy
    cherrypy.config.update({'server.socket_host': local_ip, 'server.socket_port': port})
    cherrypy.tree.mount(catalog, '/', conf)

    cherrypy.engine.start()

    print("\n_____\n")
    print(f"  Catalog running at http://{local_ip}:{port}")
    print("_____\n")
    print("Press Ctrl+C to stop the server. \n")
```

Catalog

- Stores and distributes information contained on a .json file
- Catalog .json contains information about general configuration, registered devices and registered patients

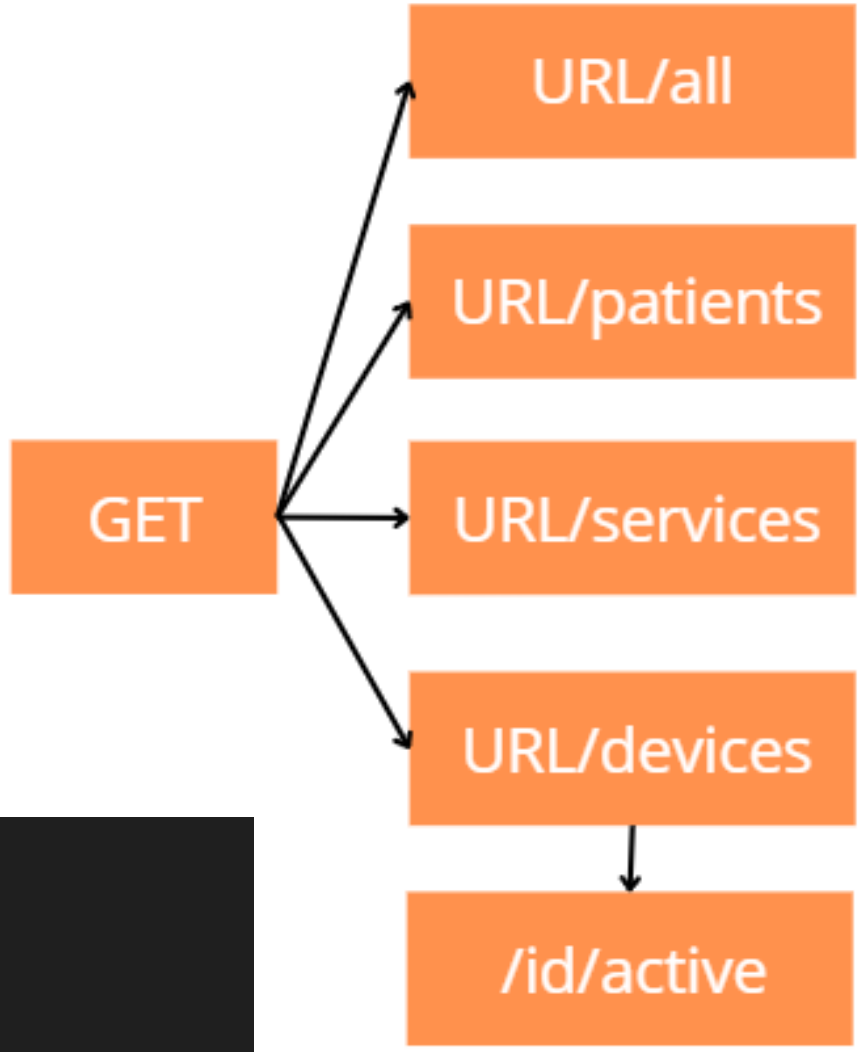
Catalog

- Handles communication with all microservices in the platform
- It accepts 4 types of requests, GET, POST, PUT and DELETE

```
> class CatalogREST: ···


def run_cherrypy_server(catalog):
    local_ip="catalog"
    port = 8080
    # Configurazioni di CherryPy
    cherrypy.config.update({'server.socket_host': local_ip, 'server.socket_port': port})
    cherrypy.tree.mount(catalog, '/', conf)

    cherrypy.engine.start()

    print("\n_____\n")
    print(f"  Catalog running at http://{local_ip}:{port}")
    print("_____\n")
    print("Press Ctrl+C to stop the server. \n")
```

Catalog

- Handles communication with all microservices in the platform
- Retrieving data ->GET

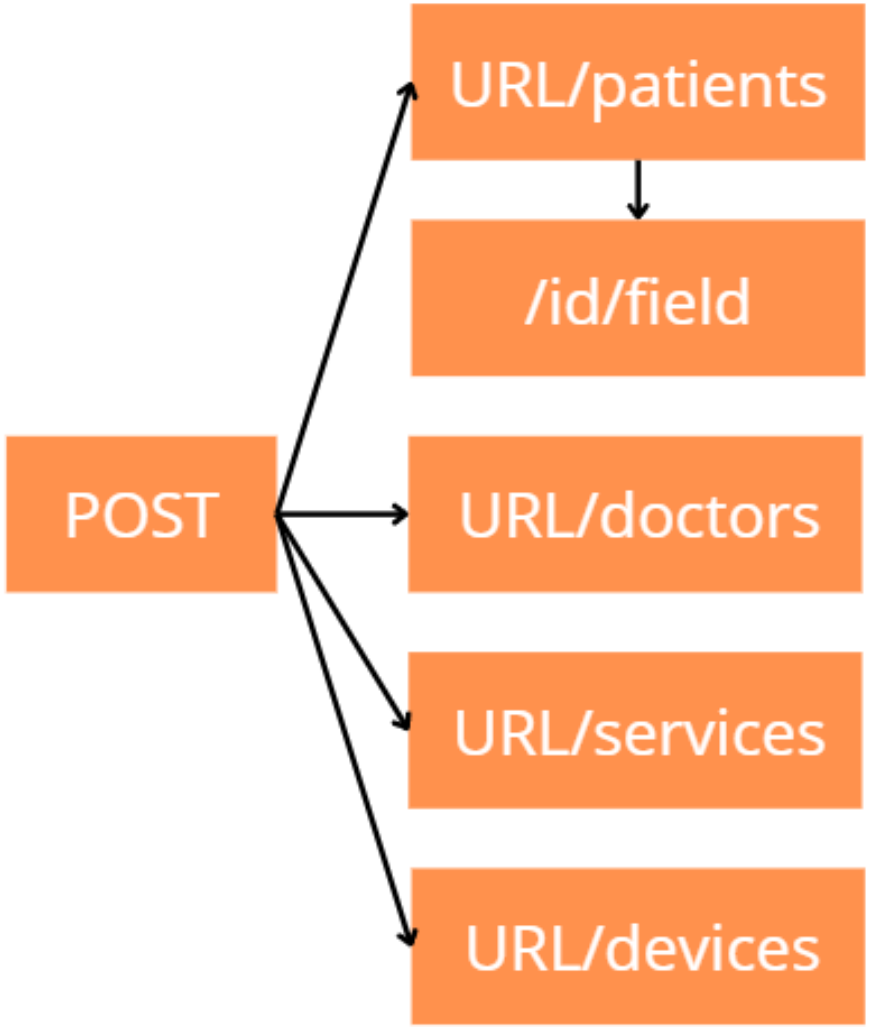



```
def GET(self, *uri, **params):
    with lock:
        with open(self.catalog_address, "r") as f:
            catalog = json.load(f)
        if len(uri)==0: #An error will be raised in case there is no uri
            raise cherrypy.HTTPError(status=400, message='UNABLE TO MANAGE THIS URL')
```

Lock: makes sure one request is taken care of at each time.

Catalog

- Handles communication with all microservices in the platform
- Adding data ->POST

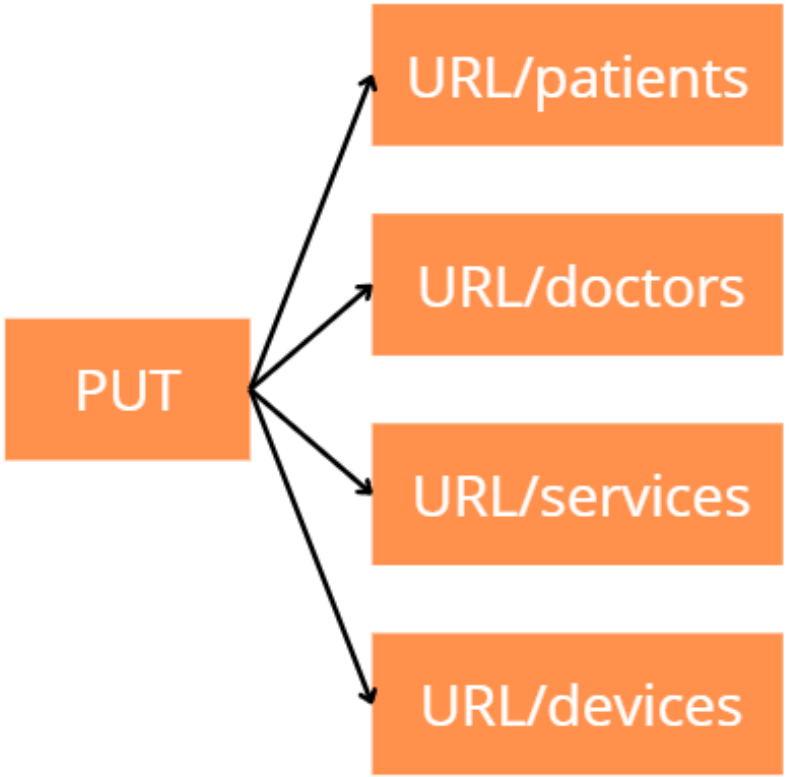



```
def POST(self, *uri, **params):
    with lock:
        with open(self.catalog_address, "r") as f:
            catalog = json.load(f)
        body = cherrypy.request.body.read()
        json_body = json.loads(body.decode('utf-8'))
```

Lock: makes sure one request is taken care of at each time.

Catalog

- Handles communication with all microservices in the platform
- Updating data ->PUT

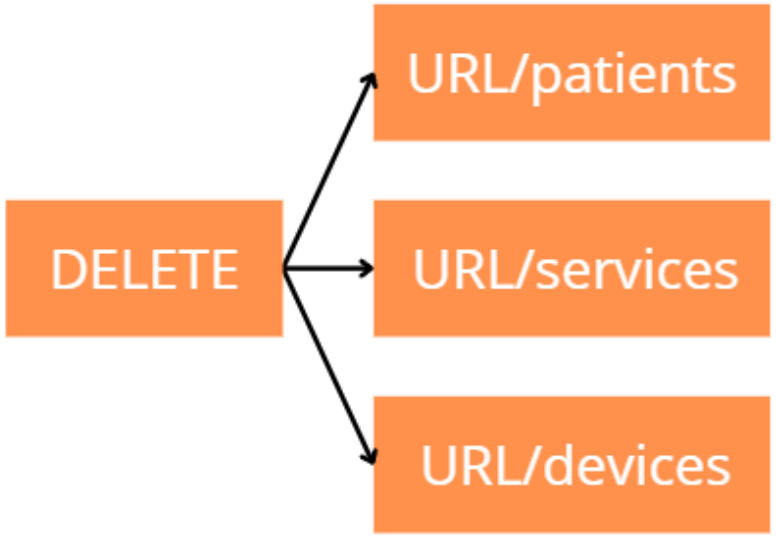



```
def PUT(self, *uri, **params):
    with lock:
        with open(self.catalog_address, "r") as f:···
        body = cherrypy.request.body.read()
        json_body = json.loads(body.decode('utf-8'))
        if uri[0]=='patients':···
        elif uri[0]=='devices':···
```

Lock: makes sure one request is taken care of at each time.

Catalog

- Handles communication with all microservices in the platform
- Deleting data ->DELETE



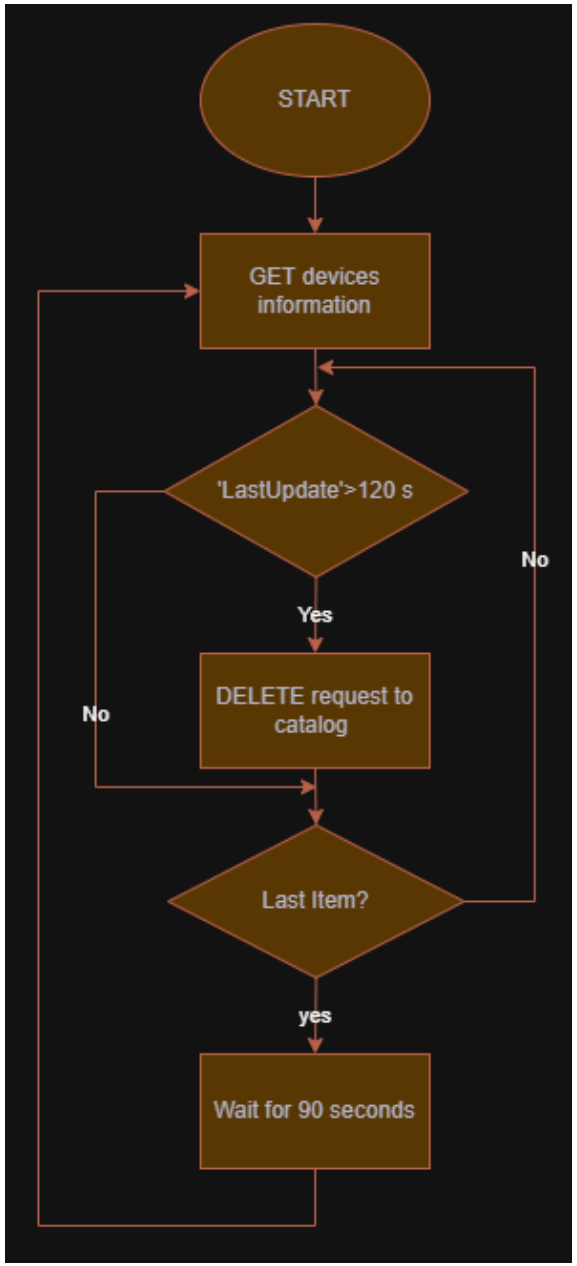

```
def DELETE(self, *uri):
    with lock:
        with open(self.catalog_address, "r") as f:···
        if uri[0]=='devices':···
        elif uri[0]=='patients':···
        elif uri[0]=='services':
```

Lock: makes sure one request is taken care of at each time.

Device Monitor

Microservice in charge of monitoring the activity of all devices

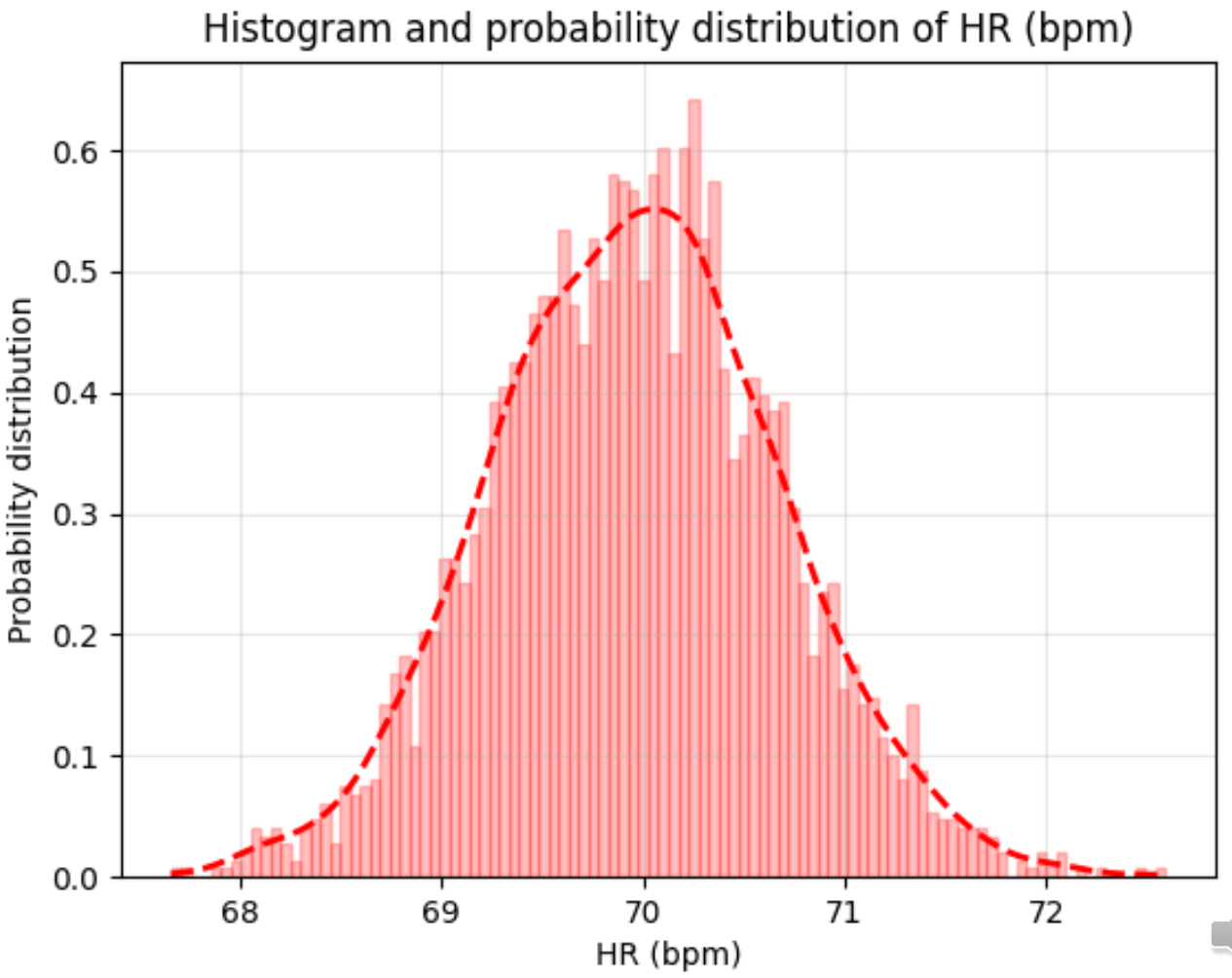
- Eliminates devices from the catalog if the last update contains a time prior to 120 seconds



Health Connector

Health connector implements two sensor simulation:

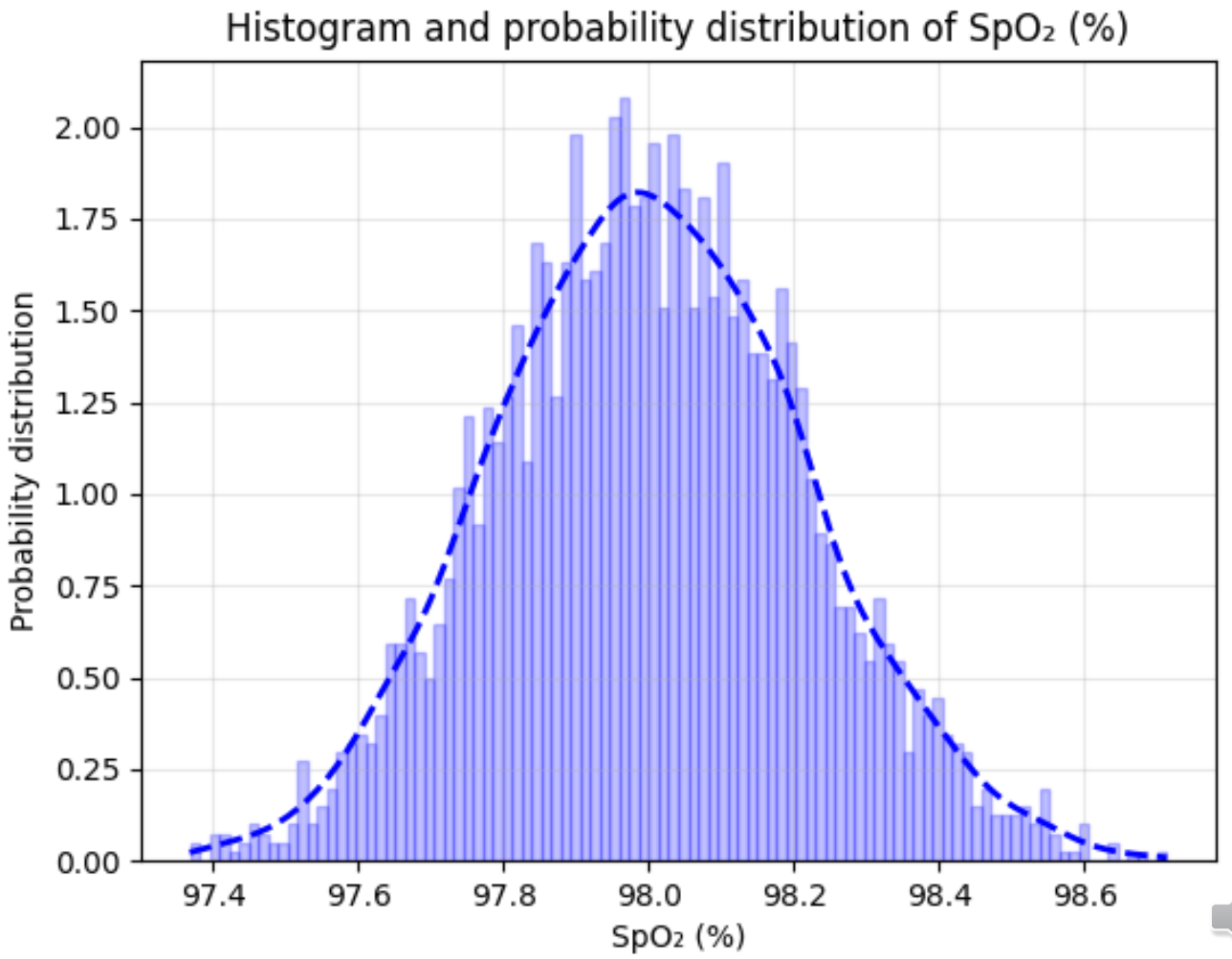
- **Heart Rate sensor**
- Blood Oxygen Saturation sensor



Health Connector

Health connector implements two sensor simulation:

- Heart Rate sensor
- **Blood Oxygen Saturation sensor**



Health Connector

Health connector communicates with other services using **SenML format.**

```
self.message = {
                "bn": f"DeviceConnector_{self.clientID}",
                "e": [
                    {
                        "n": "spo2",
                        "v": 98,
                        "t": "",
                        "u": "%"
                    },
                    {
                        "n": "hr",
                        "v": 70,
                        "t": "",
                        "u": "bpm"
                    }
                ]
            }
```

Topic: poli/IoTProject/sensors/#device_id/health

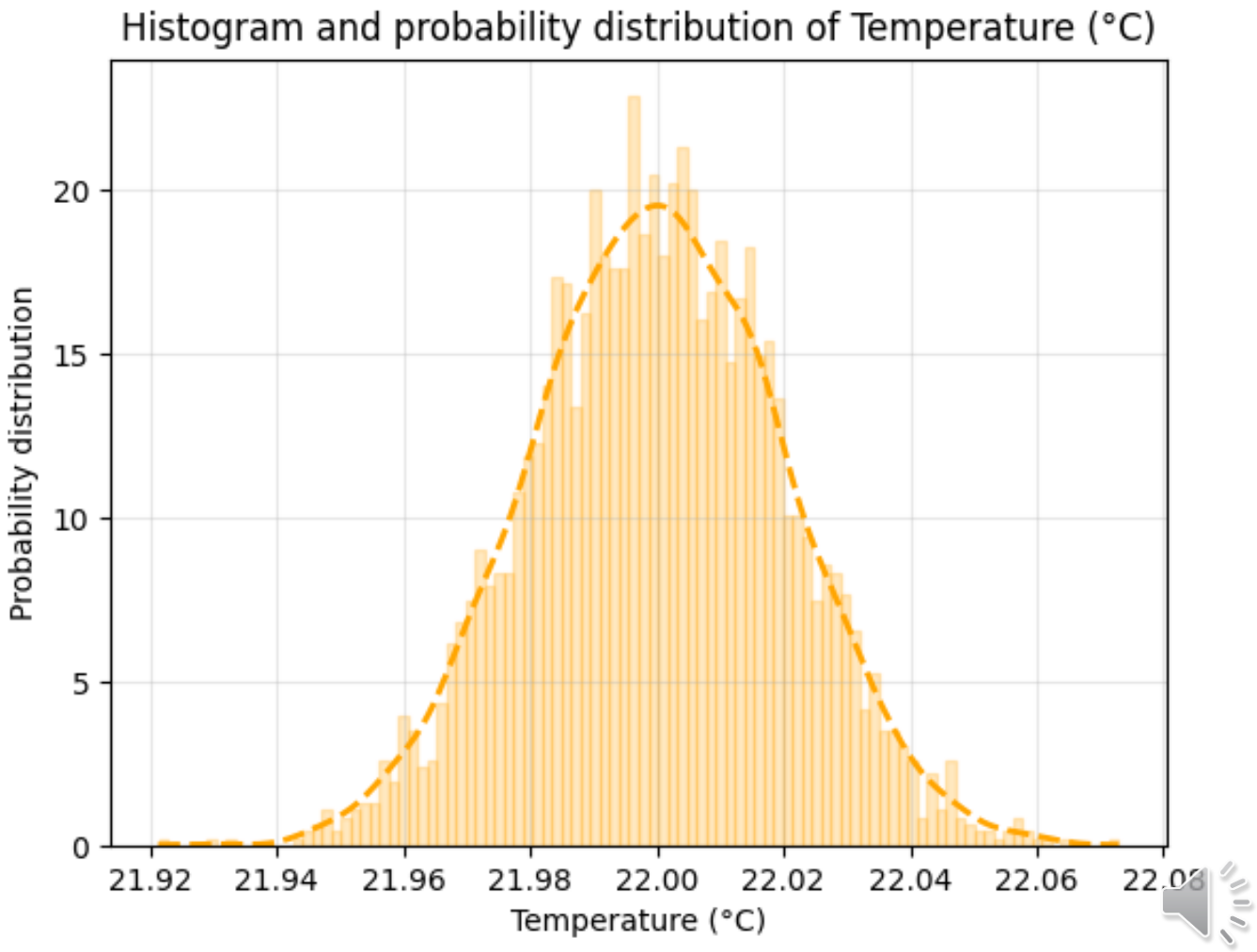
Health Connector communicates with:

- **Catalog**: to register and update the device information through REST communication.
- **ThingSpeak Adaptor**: to send recordings that will be stored in ThingSpeak through MQTT communication.

Home Connector

Home connector implements two sensor simulation:

- **Temperature sensor**
- Humidity sensor



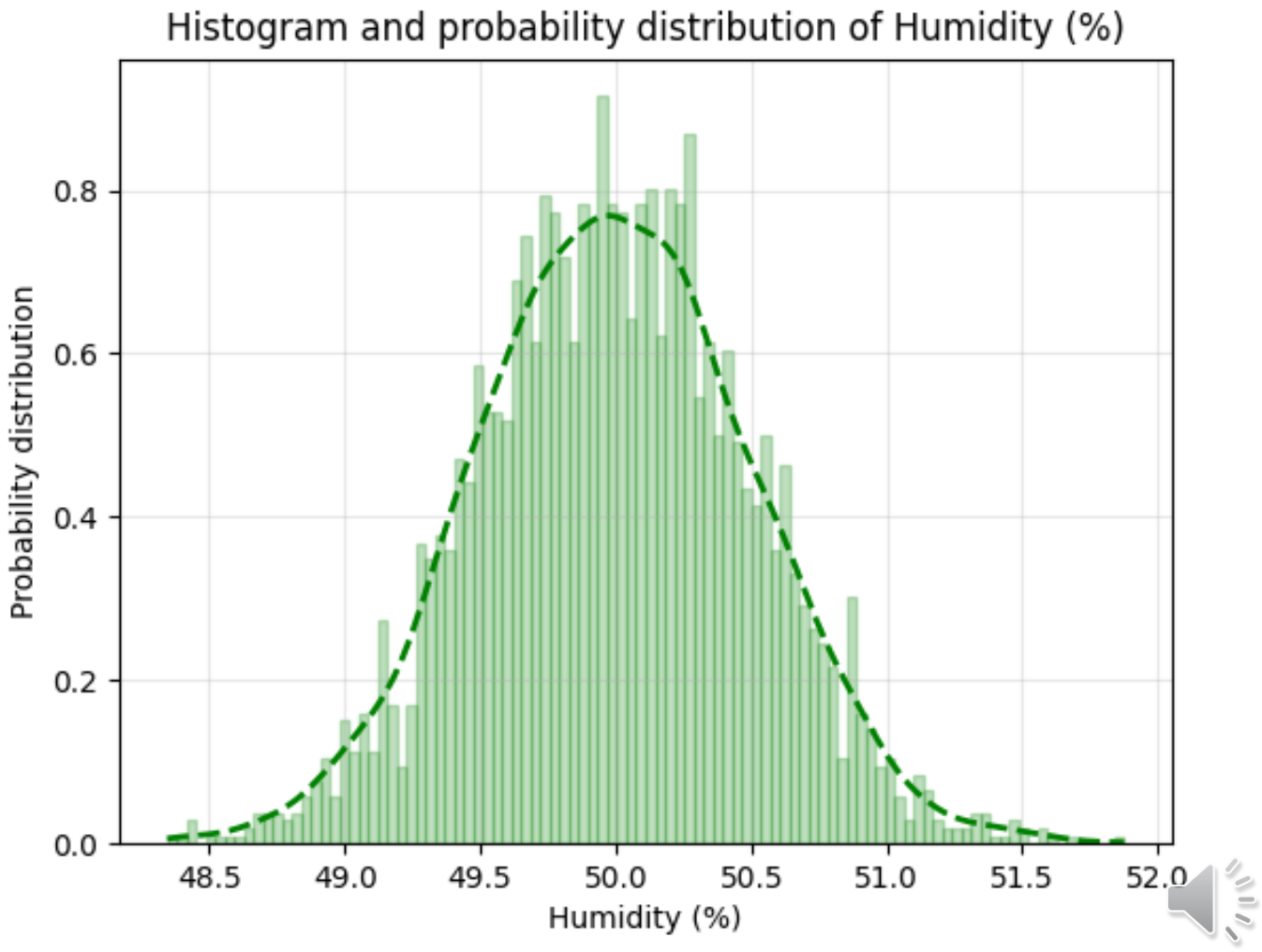
Home Connector

Home connector implements two sensor simulation:

- Temperature sensor
- **Humidity sensor**

For all simulated data:

$$x_{new} = 0.9 \cdot x_{old} + 0.1 \cdot x_{\text{generated}}$$



Home Connector

Health connector communicates with other services using **SenML format.**

```
self.message = {
                "bn": f"DeviceConnector_{self.clientID}",
                "e": [
                    {
                        "n": "temperature",
                        "v": 22.0,
                        "t": "",
                        "u": "°C"
                    },
                    {
                        "n": "humidity",
                        "v": 50.0,
                        "t": "",
                        "u": "%"
                    }
                ]
            }
```

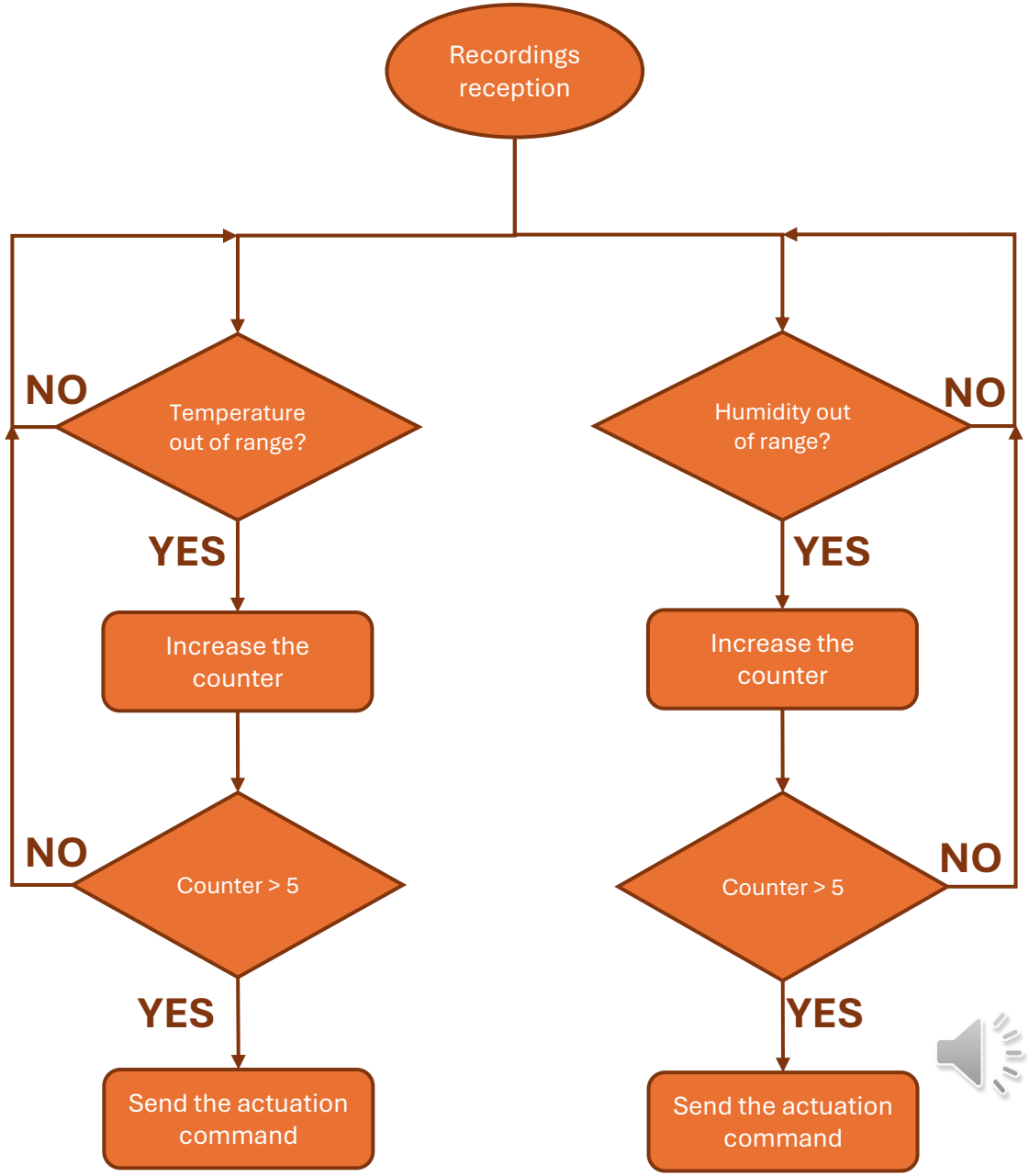
Topic: poli/IoTProject/sensors/#device_id/home

Home Connector communicates with:

- **Catalog**: to register and update the device information through REST communication.
- **ThingSpeak Adaptor**: to send recordings that will be stored in ThingSpeak through MQTT communication.
- **Timer Control:** to send recordings that will be used to evaluate the need of actuation commands.

Timer Control

Timer Control service controls that home parameters, like **temperature** and **humidity**, are within a range predefined by the patient, in order to have a comfortable environment.



Timer Control

Timer control communicates with other services using **SenML format.**

```
message = {
            "bn": f"Timercontrol",
            "e": [
                {
                    "n": parameter,
                    "v": value,
                    "t": time.time(),
                    "u": "bool"
                }
            ]
        }
```

Timer Control communicates with:

- **Catalog**: to register and update the service information through REST communication.
- **Home Connector**: to receive recordings that will be analyzed.

ThingSpeak adaptor

ThingSpeak adaptor exploits both **REST** and **MQTT** communication protocols.

REST	MQTT
Catalog	**Health Connector**
Statistic Provider	**Home connector**
ThingSpeak Database	

ThingSpeak adaptor communicates with:

- **Catalog**: to register and update the service information through REST communication.
- **Statistic Provider**: to handle data requests and return data .
- **ThingSpeak Database**: to send and store data received from the health/home connectors and retrieve data requested from the Statistic Provider.
- **Health Connector**: to receive recordings that will be stored and analyzed.
- **Home Connector**: to receive recordings that will be stored and analyzed.

Statistic Provider

The Statistic Provider is used to send a PDF file containing the requested medical data to both the doctor and the patient.

To do this, the data, are first requested from the ThingSpeak Adaptor and then processed to generate a report in PDF format.

```
class StatisticProvider:
    exposed = True
    def __init__(self, settings):...
```

Statistic Provider communicates with:

- **User interface**: communication takes place through REST APIs.
- **Catalog**: communication takes place through REST APIs.
- **ThingSpeak Adaptor:** communication takes place through REST APIs, and allows to obtain the medical data to be processed.

User Interface

As a user interface, we decided to use two Telegram bots: one for doctors and one for patients.

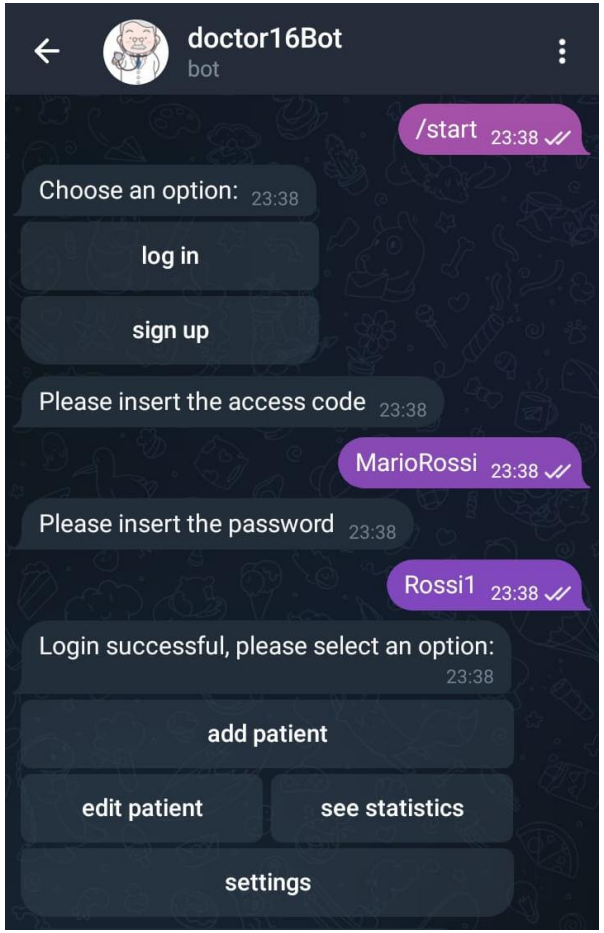
Telegram

Purpose of the two bots:

- **Doctor Bot**: it allows the doctors to register themselfs, the patient, join the patients with the devices, and see the the patient's statistics.

- **Patient Bot**: it allows the patients to see their health statistics and set their room's temperature.

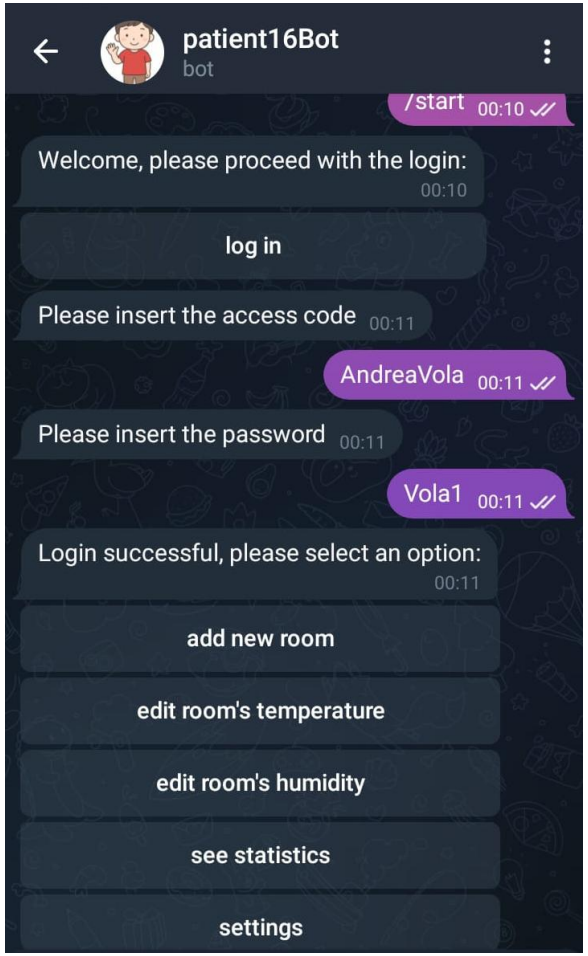
Doctor Bot



Doctor Bot communicates with:

- **Catalog**: the communication between the Catalog and the Doctor Bot occours via REST api.
- **Statistic Provider**: the communication between the Statistic Provider and the Doctor Bot occours via REST api.

Patient Bot



Patient Bot communicates with:

- **Catalog**: the communication between the Catalog and the Patient Bot occours via REST api.
- **Statistic Provider**: the communication between the Statistic Provider and the Patient Bot occours via REST api.